

L12 - Merge Sort Analysis

Monday, April 27, 2020 8:17 PM

MergeSort (Arr, low, high)	Cost	Freq.
1. if low < high {	c1	1
2. // divide problem into two smaller instances P ₁ , P ₂		
3. let mid = (low + high)/2	c2	1
4. // following two lines represent step 2 of generic DAC algorithm		
5. MergeSort(Arr, low, mid)	(?1)	1
6. MergeSort(Arr, mid+1, high)	(?2)	1
7. // step 3 of generic DAC algorithm		
8. Merge(Arr, low, mid, high)	O(n)	1
9. }		

$$T(n) = c1 + c2 + O(n) + (?1) + (?2)$$

if low < high ; if n > 1
if low = high; if n=1

$$T(n) = c1 + c2 + cn + (?1) + (?2)$$

if low < high ; if n > 1

$$= O(n) + (?1) + (?2)$$

if low = high; if n=1

$$= c1$$

$$T(n) = O(n) + T(n/2) + T(n/2)$$

if low < high ; if n > 1
if low = high; if n=1

$$= c1$$

$$T(n) = \begin{cases} c1; & \text{if } n = 1 \\ 2T(n/2) + O(n); & \text{if } n > 1 \end{cases}$$

Solving for

$$T(n) = 2T(n/2) + cn; \quad \text{if } n > 0 \quad (\text{eq. 1})$$

where c & c1 are positive constants:

Using equation 1, above, repeatedly,

$$T(n) = 2T(n/2) + cn$$

Substituting $n/2$ for n in equation 1

$$= 2[2T(n/2^2) + c(n/2)] + cn = 2^2T(n/2^2) + cn + cn$$

$$= 2^2T(n/2^2) + 2cn$$

Substituting $n/2^2$ for n in equation 1

$$= 2^2[2T(n/2^3) + c(n/2^2)] + 2cn = 2^3T(n/2^3) + cn + 2cn$$

$$= 2^3T(n/2^3) + 3cn$$

...

$$= 2^kT(n/2^k) + kcn$$

Assuming n to be exact power of 2, for simplicity sake

let $n/2^k = 1$, $n = 2^k$ and $k = \log n$

$$T(n) = nT(1) + kcn$$

$$T(n) = nc1 + cn \log n$$

$$T(n) = O(n \log n) \quad // \text{ if } n=10^6 ; n^2 = 10^{12} ; n \log n = 10^6 * \log(10^6) = 20 * 10^6$$

Plug & Play

Chug & Play

Substitution Method

Recurrence Tree Method

$$T(n) = \begin{cases} c1; & \text{if } n = 1 \\ 2T(n/2) + cn; & \text{if } n > 1 \end{cases}$$

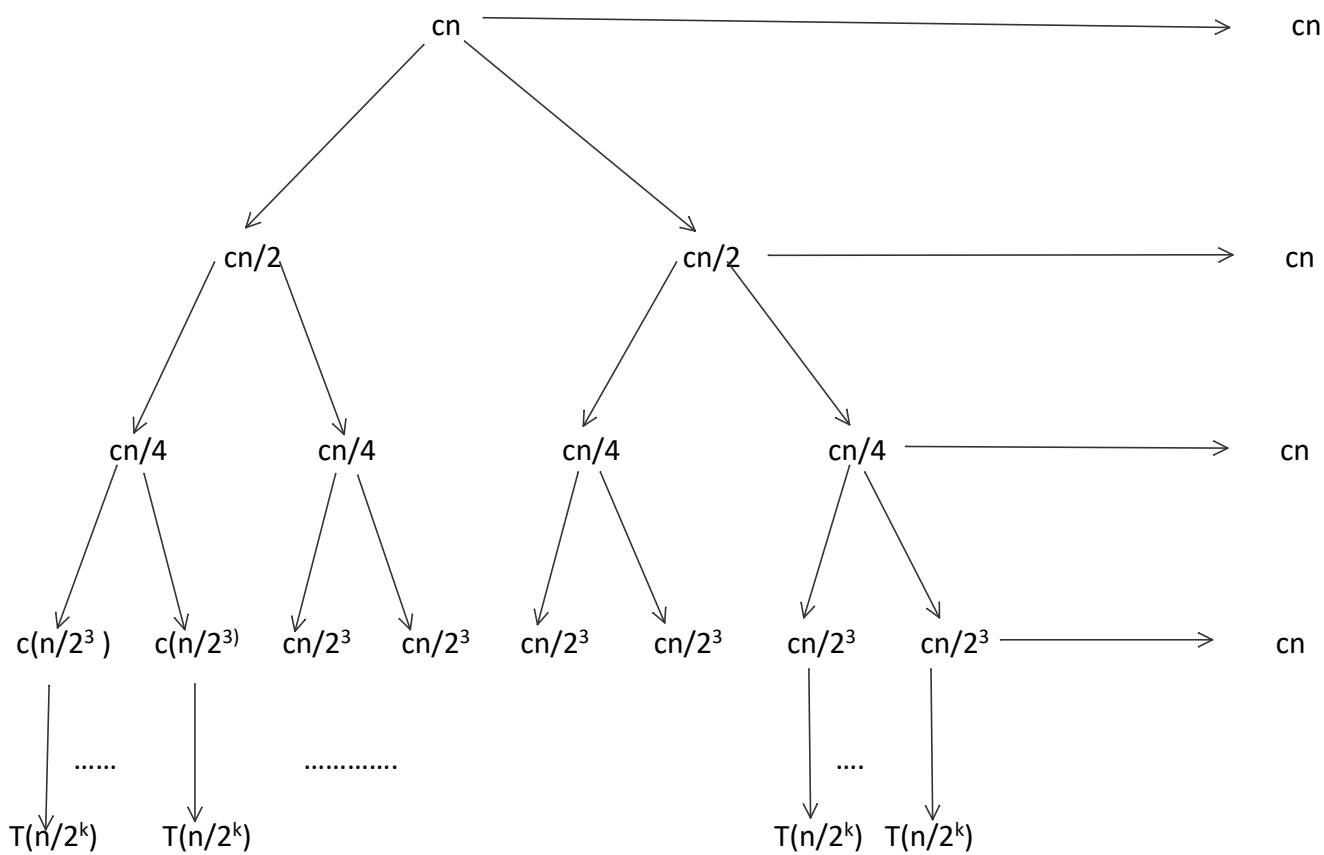
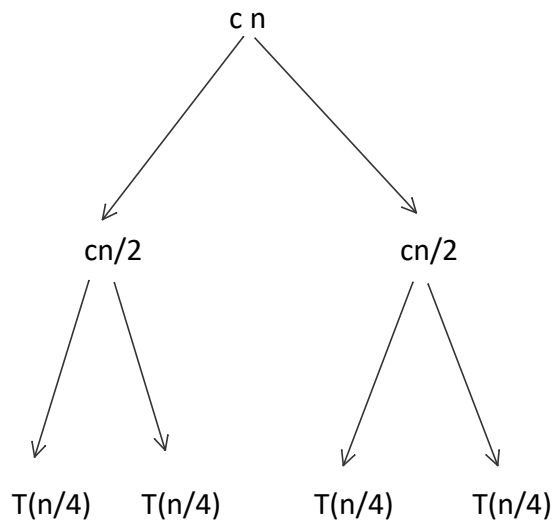
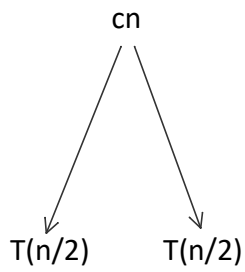
$$\text{Solving for } T(n) = 2T(n/2) + cn; \quad \text{if } n > 1$$

$T(n)$

cn
/
\

cn
/
\

$T(n)$



let $n/2^k = 1$, $n = 2^k$ and $k = \log n$
 Assuming n to be exact power of 2

$c1 \dots c1 \dots$

$c1 \dots c1 \longrightarrow c1n$

$kcn + c1n =$
 $O(n \log n)$

$cn \log n + c1n$